

Task : Monitoring User Activities

Submitted by: Hiya Sharma (21BCY10078)

Tool used: Splunk

Introduction:

It is important to create a secure environment for organizations. User activities play a crucial role in the security posture of any enterprise, monitoring user activities provides invaluable insights into system usage, behavior patterns, and potential security risks.

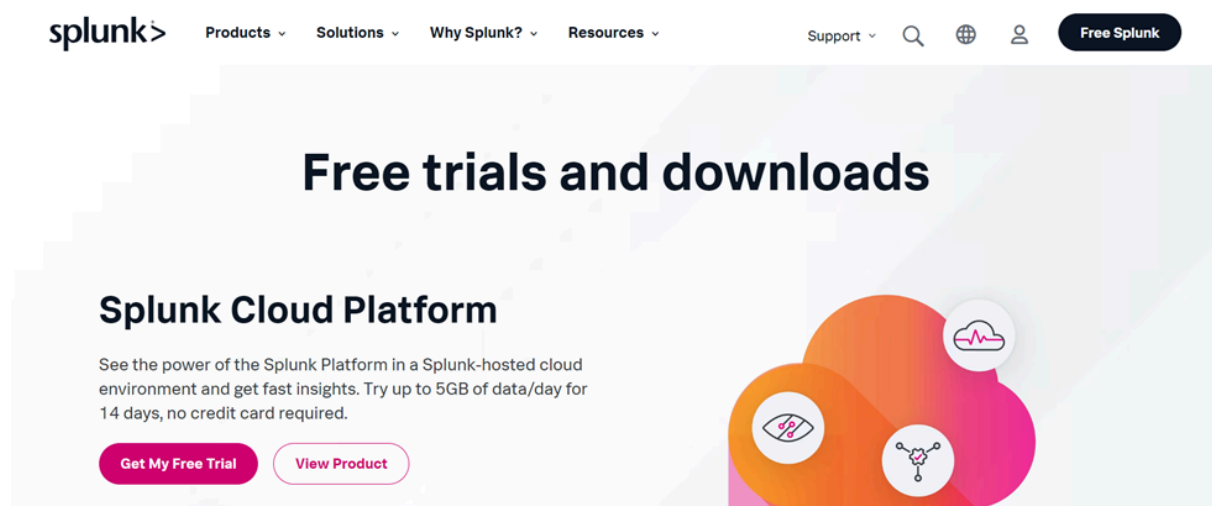
Splunk, a leading platform for collecting, analyzing, and visualizing machine-generated data, offers powerful capabilities for monitoring user activities within an organization's IT infrastructure. By leveraging Splunk's robust features and functionalities, organizations can gain visibility into user interactions with critical systems, applications, and data assets.

Through comprehensive data collection, advanced analytics, and real-time monitoring, Splunk empowers organizations to proactively detect and mitigate security threats, enforce compliance requirements, and optimize operational efficiency.

Installation and setup:

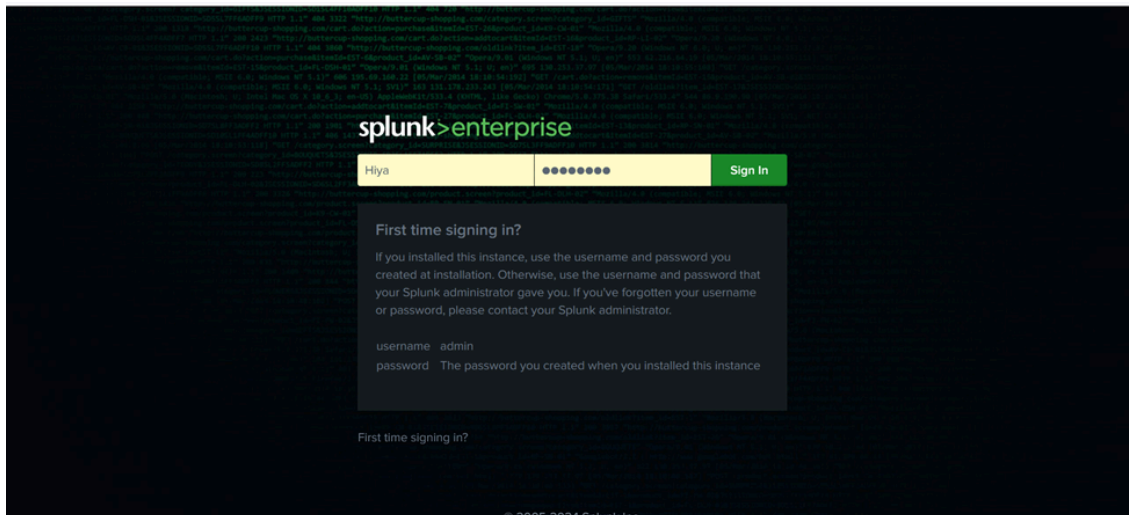
Before installing Splunk, ensure that your system meets the minimum requirements specified by Splunk. This includes hardware specifications, supported operating systems, and other prerequisites.

Visit the Splunk website (<https://www.splunk.com/>) and navigate to the downloads section.

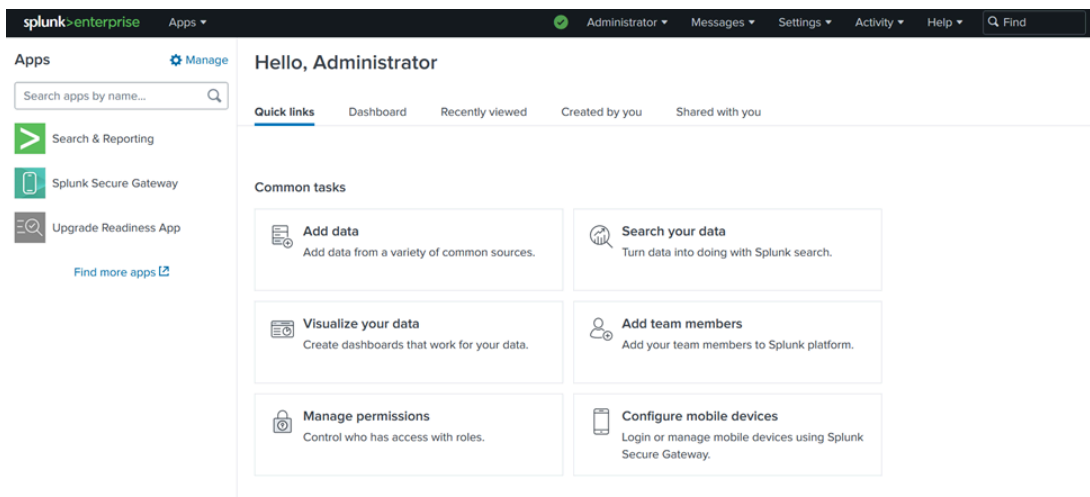


Choose the appropriate version of Splunk for your operating system and download the installer package. Run the downloaded installer package to begin the installation process.

Follow the on-screen instructions to install Splunk. You may need to specify installation directories, accept license agreements, and configure other settings during the installation.



Once the installation is complete, start Splunk by running the Splunk service or executable file. Open a web browser and navigate to the Splunk web interface. By default, Splunk's web interface is accessible at <http://localhost:8000/>. Once Splunk is set up, you can start exploring its features and functionalities. You can ingest data from various sources, create searches and reports, build dashboards, and more.



Monitoring user activities using Splunk

Step 1: Set Up Data Collection

Identify Data Sources:

1. Determine which logs and data sources you need to monitor user activities. Common sources include:
 - Authentication logs (e.g., Windows Event Logs, syslog from Linux systems)
 - Network traffic logs (firewall logs, proxy logs)
 - Endpoint logs (antivirus logs, endpoint detection logs)

Configure Splunk Forwarders:

2. Install and configure Splunk Forwarders on the systems that generate these logs. Forwarders will collect and send logs to your Splunk deployment.

Authentication logs

Forwarding windows logs to splunk using splunk universal forwarder:

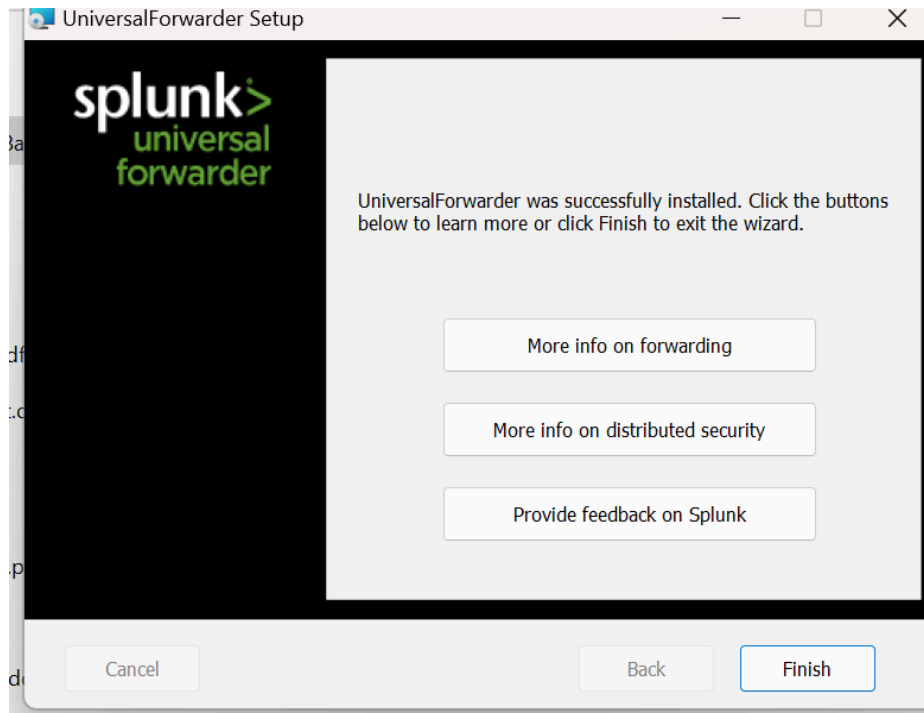
During the setup of the Splunk Universal Forwarder on your Windows system, you will need to specify:

1. Identify Splunk Indexer's Location:

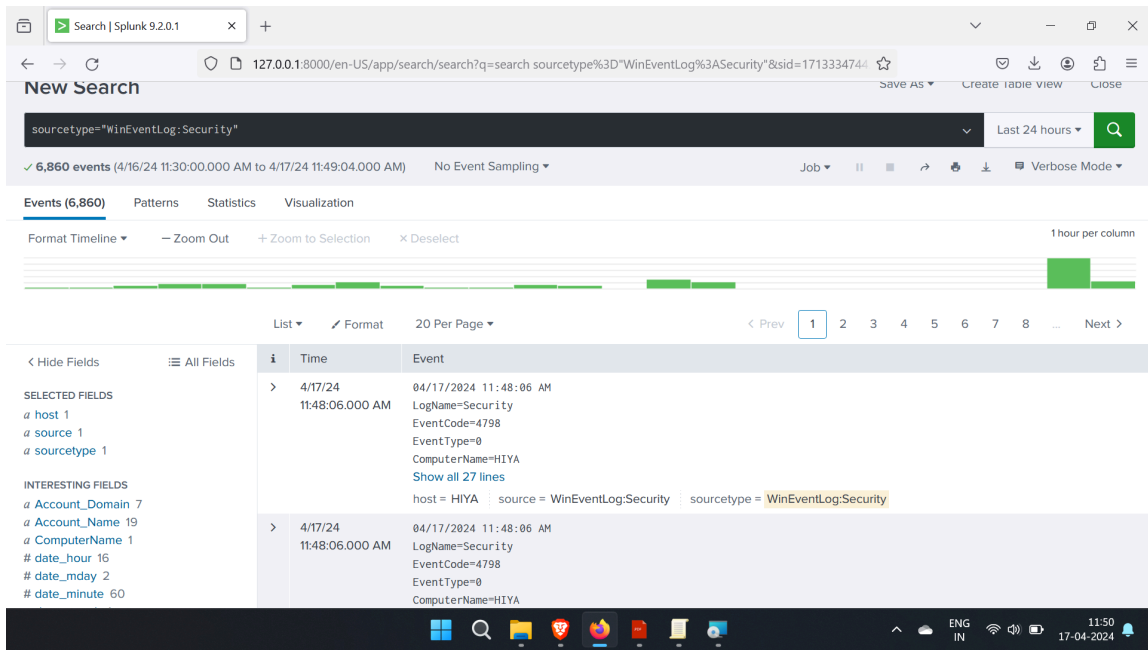
- Find out the IP address or hostname of the machine where your Splunk indexer is installed and running.

2. Determine Splunk Indexer receiving Port:

- Check the port number configured on your Splunk indexer for receiving data. The default port for receiving data from forwarders is typically 9997 for Splunk TCP input.

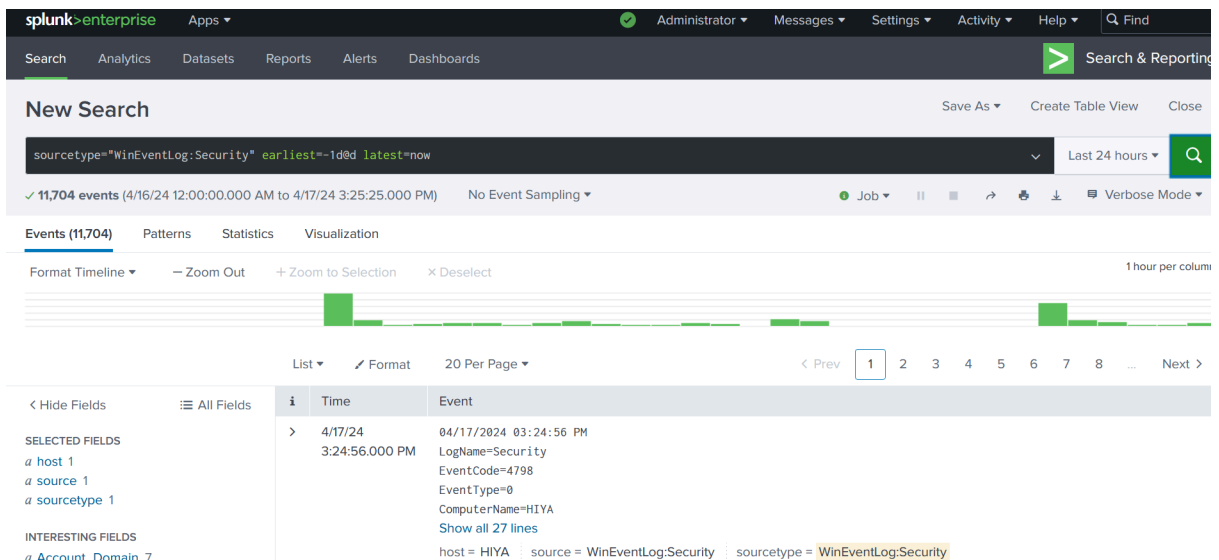


Windows Security Event Logs:



- Splunk will display search results based on your query, showing the Windows logs that have been forwarded and indexed.

Filter by Time Range:



Common Security Event IDs

- Failed Logon Attempts: Event ID 4625 (Security Log)
- Successful Logon Events: Event ID 4624 (Security Log)

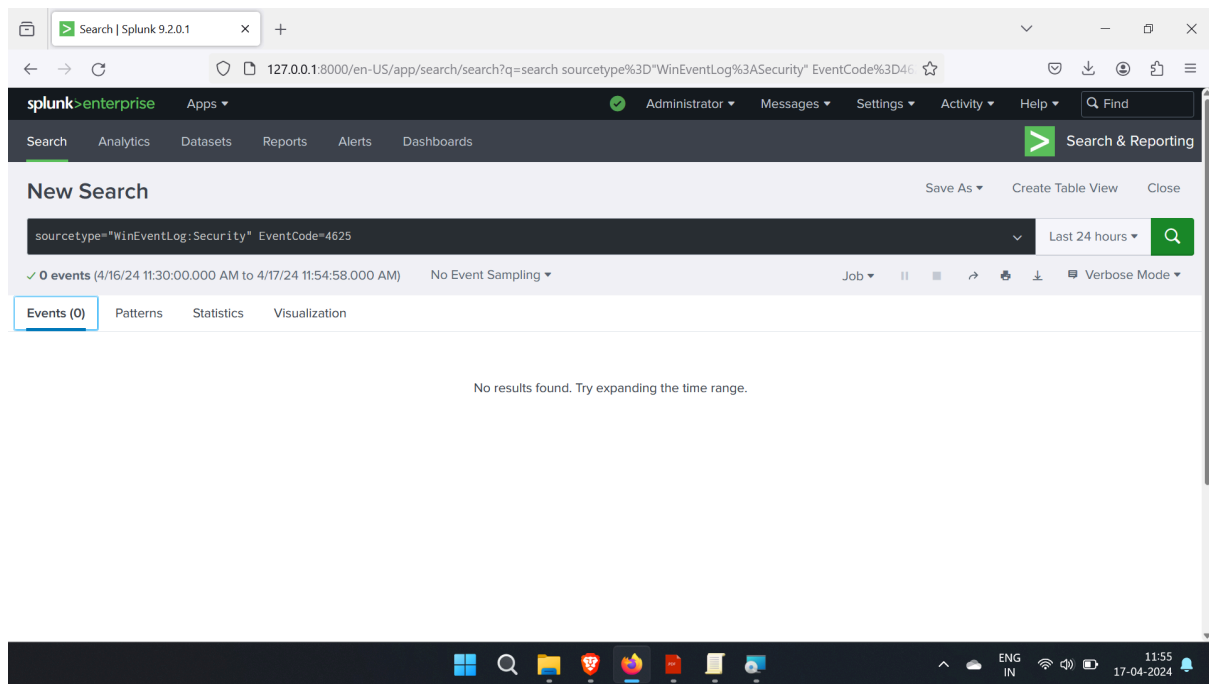
- Privilege Escalation: Event ID 4672 (Security Log)
- File Access or Modification: Event ID 4663 (Security Log)
- Process Creation: Event ID 4688 (Security Log)
- Account Management: Various Event IDs in the Security Log

Construct Search Queries for Suspicious Activities:

Example Queries:

1.Failed Logon Attempts:

`sourcetype="WinEventLog:Security" EventCode=4625`



2.Successful Logon Events from Unusual Locations:

`sourcetype="WinEventLog:Security" EventCode=4624`

`| stats count by src_ip`

`| where count > 100`

New Search

sourcetype="WinEventLog:Security" EventCode=4624
 | stats count by src_ip
 | where count > 100

✓ 261 events (4/16/24 11:30:00.000 AM to 4/17/24 11:53:18.000 AM) No Event Sampling

Events (261) Patterns Statistics (0) Visualization

Format Timeline Zoom Out Zoom to Selection Deselect 1 hour per column

List Format 20 Per Page

Time	Event
4/17/24 11:46:36.000 AM	04/17/2024 11:46:36 AM LogName=Security EventCode=4624 EventType=0 ComputerName=HIYA SourceName=Microsoft Windows security auditing. Type=Information RecordNumber=587813 Keywords=Audit Success TaskCategory=Logon OpCode=Info

3.Detect Privilege Escalation:

sourcetype="WinEventLog:Security" EventCode=4672
 | search "New Process Name"="*\cmd.exe"

New Search

sourcetype="WinEventLog:Security" EventCode=4672
 | search "New Process Name"="*\cmd.exe"

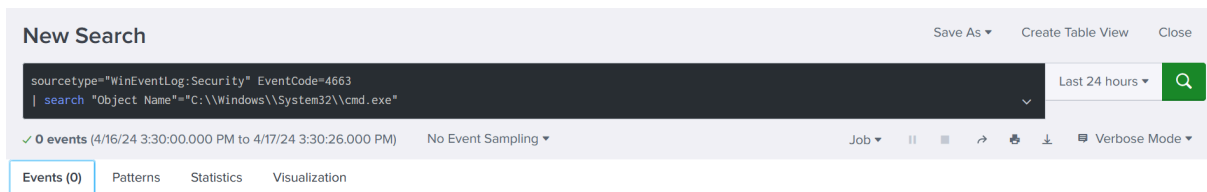
✓ 0 events (4/16/24 11:30:00.000 AM to 4/17/24 11:55:18.000 AM) No Event Sampling

Events (0) Patterns Statistics Visualization

No results found. Try expanding the time range.

4.File Access or Modification:

sourcetype="WinEventLog:Security" EventCode=4663
 | search "Object Name"="C:\\Windows\\System32\\cmd.exe"



No results found. Try expanding the time range.

4. Create Real-Time Alerts

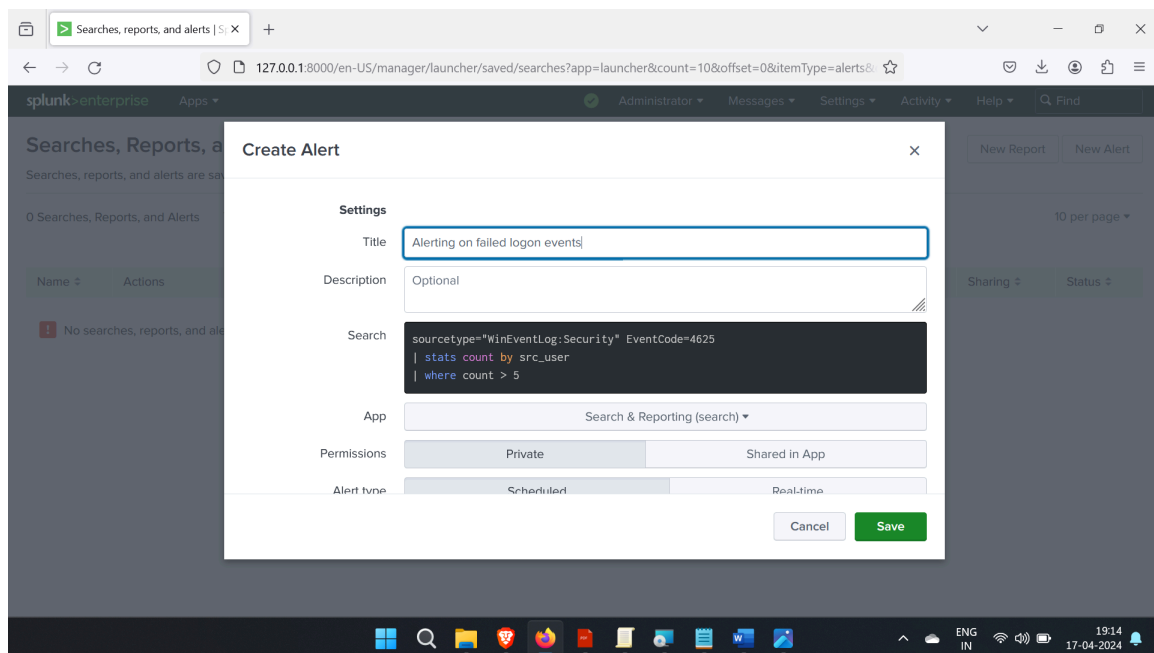
Set up real-time alerts based on specific search criteria to receive notifications for suspicious activities as they occur:

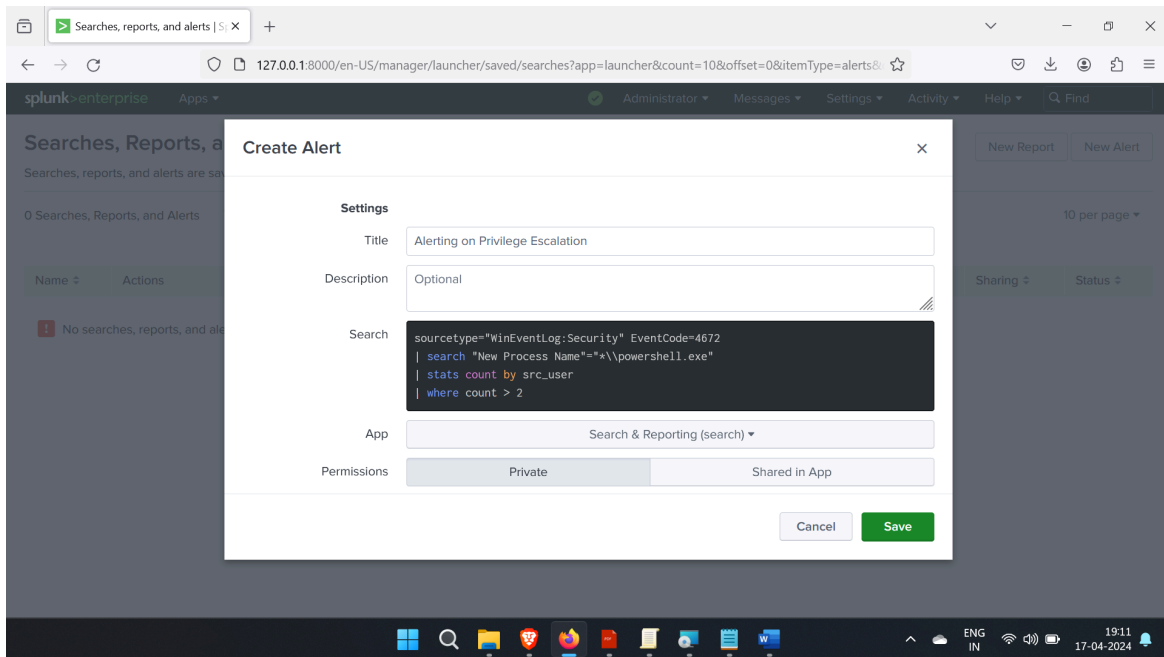
Alerting on Failed Logon Attempts:

```
sourcetype="WinEventLog:Security" EventCode=4625
| stats count by src_user
| where count > 5
```

Alerting on Privilege Escalation:

```
sourcetype="WinEventLog:Security" EventCode=4672
| search "New Process Name"="*\\powershell.exe"
| stats count by src_user
| where count > 2
```





Step 5: Monitor Real-Time Alerts

1. Access Alert Manager:

- Navigate back to the "Settings" > "Searches, reports, and alerts" section to view and manage your configured alerts.

2. Monitor Alert Status:

- Monitor the status and execution of your real-time alerts to track detected suspicious activities.

Monitoring Windows Firewall Logs:

Step 1: Understanding Windows Firewall Logs

Windows Firewall logs are stored in text files with a .log extension in the directory %SystemRoot%\system32\LogFiles\Firewall. Each log entry contains information about network traffic, including details like timestamp, source IP, destination IP, protocol, action (allowed or blocked), etc.

Step 2: Reading and Parsing Log Entries

Parse the log entries to extract relevant information.

Code:

```

def parse_firewall_logs(log_file_path):
    log_entries = []

    try:
        with open(log_file_path, 'r') as file:
            for line in file:
                if line.startswith('#'):
                    continue

                fields = line.strip().split('\t')

                if len(fields) >= 15:
                    log_entry = {
                        'Date': fields[0],
                        'Time': fields[1],
                        'Action': fields[2],
                        'Protocol': fields[3],
                        'SourceIP': fields[4],
                        'DestinationIP': fields[5],
                        'SourcePort': fields[6],
                        'DestinationPort': fields[7],
                        'Size': fields[8],
                        'TCPFlags': fields[9],
                        'TCPSYN': fields[10],
                        'TCPACK': fields[11],
                        'TCPWin': fields[12],
                        'ICMPType': fields[13],
                        'ICMPCode': fields[14]
                    }
                    log_entries.append(log_entry)

    except FileNotFoundError:
        print(f"Log file '{log_file_path}' not found.")

    return log_entries

desktop_path = "C:\\Users\\hiyas\\OneDrive\\Desktop\\" # Update with your desktop path
firewall_log_path = desktop_path + "pfirewall.log1.txt"

parsed_logs = parse_firewall_logs(firewall_log_path)

for log in parsed_logs:
    print(log)

```

Output:

```
IDLE Shell 3.12.3
File Edit Shell Debug Options Window Help
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\workarea\read_parseFirewall_Logs.py
{'Date': 'Name', 'Time': 'Profile', 'Action': 'Action', 'Protocol': 'Override', 'SourceIP': 'Direction', 'DestinationIP': 'Program', 'SourcePort': 'Local Address', 'DestinationPort': 'Remote Address', 'Size': 'Protocol', 'TCPFlags': 'Local Port', 'TCPSYN': 'Remote Port', 'TCPACK': 'Authorized Users', 'TCPWin': 'Authorized Computers', 'ICMPType': 'Authorized Local Principals', 'ICMPCode': 'Local User Owner'}
{'Date': '@{C27EB4BA.DropboxOEM_20.4.8.0_x64_xbfy0k16fey96?ms-resource://C27EB4BA.DropboxOEM/Resources/OEMAppName}', 'Time': 'All', 'Action': 'Allow', 'Protocol': 'No', 'SourceIP': 'Inbound', 'DestinationIP': 'Any', 'SourcePort': 'Any', 'DestinationPort': 'Any', 'Size': 'Any', 'TCPFlags': 'Any', 'TCPSYN': 'Any', 'TCPACK': 'Any', 'TCPWin': 'Any', 'ICMPType': 'Any', 'ICMPCode': 'HIYA\\Administrator'}
{'Date': '@{Microsoft.Windows.Photos_21.21030.25003.0_x64_8wekyb3d8bbwe?ms-resource://Microsoft.Windows.Photos/Resources/AppStoreName}', 'Time': 'All', 'Action': 'Allow', 'Protocol': 'No', 'SourceIP': 'Inbound', 'DestinationIP': 'Any', 'SourcePort': 'Any', 'DestinationPort': 'Any', 'Size': 'Any', 'TCPFlags': 'Any', 'TCPSYN': 'Any', 'TCPACK': 'Any', 'TCPWin': 'Any', 'ICMPType': 'Any', 'ICMPCode': 'HIYA\\Administrator'}
{'Date': '@{microsoft.windowscommunicationsapps_16005.14326.20544.0_x64_8wekyb3d8bbwe?ms-resource://microsoft.windowscommunicationsapps/hxoutlookintl/AppManifest_OutlookDesktop_DisplayName}', 'Time': 'All', 'Action': 'Allow', 'Protocol': 'No', 'SourceIP': 'Inbound', 'DestinationIP': 'Any', 'SourcePort': 'Any', 'DestinationPort': 'Any', 'Size': 'Any', 'TCPFlags': 'Any', 'TCPSYN': 'Any', 'TCPACK': 'Any', 'TCPWin': 'Any', 'ICMPType': 'Any', 'ICMPCode': 'HIYA\\Administrator'}
{'Date': '@{Microsoft.XboxGamingOverlay_2.622.3232.0_x64_8wekyb3d8bbwe?ms-resource://Microsoft.XboxGamingOverlay/resources/GameBar}', 'Time': 'All', 'Action': 'Allow', 'Protocol': 'No', 'SourceIP': 'Inbound', 'DestinationIP': 'Any', 'SourcePort': 'Any', 'DestinationPort': 'Any', 'Size': 'Any', 'TCPFlags': 'Any', 'TCPSYN': 'Any', 'TCPACK': 'Any', 'TCPWin': 'Any', 'ICMPType': 'Any', 'ICMPCode': 'HIYA\\Administrator'}
{'Date': 'allow syslog udp port', 'Time': 'All', 'Action': 'Allow', 'Protocol': 'No', 'SourceIP': 'Inbound', 'DestinationIP': 'Any', 'SourcePort': 'Any', 'DestinationPort': 'Any', 'Size': 'UDP', 'TCPFlags': '514', 'TCPSYN': 'Any', 'TCPACK': 'Any'}
```

Step 3: Monitoring Logs for Real-Time Updates

We'll implement a mechanism to continuously monitor these log files for new entries. This can be achieved using a loop that checks for updates at regular intervals.

code:

```
monitorLogs.py - C:/workarea/monitorLogs.py (3.12.3)
File Edit Format Run Options Window Help

def monitor_firewall_log(log_file_path):
    try:
        if not os.path.exists(log_file_path):
            print(f"Error: Log file '{log_file_path}' not found.")
            return

        print(f"Monitoring firewall log file: {log_file_path}")

        # Get initial file size
        current_file_size = os.path.getsize(log_file_path)

        while True:
            # Get current file size
            new_file_size = os.path.getsize(log_file_path)

            # Check if the file size has increased (new entries added)
            if new_file_size > current_file_size:
                print("New entries detected!")

                # Open the log file and read new entries
                with open(log_file_path, 'r') as log_file:
                    log_file.seek(current_file_size) # Move to the last read position
                    new_entries = log_file.read(new_file_size - current_file_size)

                # Display new entries
                print("New Entries:")
                print(new_entries)

                # Update current file size
                current_file_size = new_file_size

            # Sleep for a short interval (e.g., 5 seconds) before checking again
            time.sleep(5)

    except Exception as e:
        print(f"Error: An unexpected error occurred: {e}")

if __name__ == "__main__":
    log_file_path = r'C:\Users\hiyas\OneDrive\Desktop\pfirewall.log1.txt'
    monitor_firewall_log(log_file_path)
```

Output:

```
>>>
===== RESTART: C:/workarea/monitorLogs.py =====
Monitoring firewall log file: C:\Users\hiyas\OneDrive\Desktop\pfirewall.log1.txt
```

To test the log monitoring script manually, we can simulate changes in the **pfirewall.log1.txt** file by adding new entries or modifying existing entries.

For example: we add-

2024-05-07 14:30:00 Firewall rule triggered: Block IP address 192.168.1.100
2024-05-07 14:31:00 Connection attempt from 10.0.0.5 blocked

File	Edit	View												
Riot Client	Public	Allow	No	Inbound	C:\riot games\riot client\riotclientelectron\riot client.exe	Any	Any	Any	Any	None	Any	OK		
Any	Any	TCP	Any	Any	Any	Any	Any	Any	Any	None	Any	OK		
Riot Client	Public	Allow	No	Inbound	C:\riot games\riot client\riotclientelectron\riot client.exe	Any	Any	Any	Any	None	Any	OK		
Any	Any	UDP	Any	Any	Any	Any	Any	Any	Any	None	Any	OK		
WFD ASP Coordination Protocol (UDP-In)	Local subnet	UDP	7235	7235	Any	Any	Any	Any	Any	None	Any	WLAN Service - WFD		
Application Services Platform Coordination Protocol (Uses UDP)														
Windows Mixed Reality	All	Allow	No	Inbound	Any	Any	Any	Any	Any	Any	Any	Any		
Any	Any	HIYA\hiyas	None	HoloShell_cw5n1h2txyewy	Windows Mixed Reality	OK								
Wireless Display Infrastructure Back Channel (TCP-In)				All	Allow	No	Inbound	C:\windows\system32\svchost.exe	Any					
\CastSrv.exe	Any	Any	TCP	7250	Any	Any	Any	Any	Any	None	Any	Wireless		
Display OK														
World Wide Web Services (HTTP Traffic-In)				All	Allow	No	Inbound	System	Any	Any	TCP	80		
Any	Any	Any	Any	Any	None	Any	World Wide Web Services (HTTP)	OK						
World Wide Web Services (HTTPS Traffic-In)				All	Allow	No	Inbound	System	Any	Any	TCP			
443	Any	Any	Any	Any	Any	None	Any	Secure World Wide Web Services (HTTPS)	OK					
World Wide Web Services (QUIC Traffic-In)				All	Allow	No	Inbound	System	Any	Any	UDP			
443	Any	Any	Any	Any	Any	None	Any	Secure World Wide Web Services (QUIC)	OK					
2024-05-07 14:30:00 Firewall rule triggered: Block IP address 192.168.1.100														
2024-05-07 14:31:00 Connection attempt from 10.0.0.5 blocked														

As i saved the file, it shows now

```
m', 'SourcePort': 'Any', 'DestinationPort': 'Any', 'Size': 'TCP', 'TCPFlags': '443', 'TCPSY',
, 'ICMPCode': 'Any'}
{'Date': 'World Wide Web Services (QUIC Traffic-In)', 'Time': 'All', 'Action': 'Allow', 'P',
, 'SourcePort': 'Any', 'DestinationPort': 'Any', 'Size': 'UDP', 'TCPFlags': '443', 'TCPSY',
'ICMPCode': 'Any'}

===== RESTART: C:/workarea/monitorLogs.py =====
Monitoring firewall log file: C:\Users\hiyas\OneDrive\Desktop\pfirewall.log1.txt
New entries detected!
New Entries:
2024-05-07 14:30:00 Firewall rule triggered: Block IP address 192.168.1.100
2024-05-07 14:31:00 Connection attempt from 10.0.0.5 blocked
```

Adding more

```
Monitoring firewall log file: C:\Users\hiyas\OneDrive\Desktop\pfirewall.log1.txt
New entries detected!
New Entries:
2024-05-07 14:30:00 Firewall rule triggered: Block IP address 192.168.1.100
2024-05-07 14:31:00 Connection attempt from 10.0.0.5 blocked

New entries detected!
New Entries:

"2024-05-01 10:15:30 Allow TCP 192.168.1.100 8.8.8.8 1234 80 1024 SYN ACK 8192 ICMP N/A",
"2024-05-01 10:20:45 Block UDP 10.0.0.5 8.8.4.4 5000 53 512 Any Any Any Any Any",
"2024-05-01 10:25:55 Allow ICMP 192.168.1.50 8.8.8.8 Any Any Any Any Any Any"
```

Step 4: Displaying Alerts

Based on the log entries, you can choose to display them, save them to a database, generate alerts, or take other actions.

File Edit Format Run Options Window Help

```
import os
import time

def parse_firewall_log_entry(log_entry):
    # Function to parse a single firewall log entry into a dictionary of fields
    fields = [
        'Date', 'Time', 'Action', 'Protocol', 'SourceIP', 'DestinationIP',
        'SourcePort', 'DestinationPort', 'Size', 'TCPFlags', 'TCPSYN', 'TCPACK',
        'TCPWin', 'ICMPType', 'ICMPCode'
    ]
    return dict(zip(fields, log_entry.split()))

def is_suspicious_entry(log_entry):
    # Function to determine if a firewall log entry is suspicious
    # Example criteria: Action is 'Block' and Protocol is 'UDP'
    return log_entry['Action'] == 'Block' and log_entry['Protocol'] == 'UDP'

def generate_alert(log_entry):
    # Function to generate alerts for suspicious firewall log entries
    source_ip = log_entry['SourceIP']
    destination_ip = log_entry['DestinationIP']
    print(f"ALERT: Detected suspicious traffic from {source_ip} to {destination_ip}")

def monitor_firewall_log(log_file_path):
    try:
        if not os.path.exists(log_file_path):
            print(f"Error: Log file '{log_file_path}' not found.")
            return

        print(f"Monitoring firewall log file: {log_file_path}")

        # Get initial file size
        current_file_size = os.path.getsize(log_file_path)

        while True:
            # Get current file size
            new_file_size = os.path.getsize(log_file_path)

            # Check if the file size has increased (new entries added)
            if new_file_size > current_file_size:
                print("New entries detected!")
```

```

# Open the log file and read new entries
with open(log_file_path, 'r') as log_file:
    log_file.seek(current_file_size) # Move to the last read position
    new_entries = log_file.read(new_file_size - current_file_size)

# Split new entries into lines and parse each log entry
for line in new_entries.splitlines():
    parsed_entry = parse_firewall_log_entry(line)

    # Check if the entry is suspicious
    if is_suspicious_entry(parsed_entry):
        generate_alert(parsed_entry) # Generate alert for suspicious entry

    # Print all new entries (for demonstration purposes)
    print("New Entry:")
    print(line)

# Update current file size
current_file_size = new_file_size

# Sleep for a short interval (e.g., 5 seconds) before checking again
time.sleep(5)

except Exception as e:
    print(f"Error: An unexpected error occurred: {e}")

if __name__ == "__main__":
    log_file_path = r'C:\Users\hiyas\OneDrive\Desktop\pfirewall.log1.txt'
    monitor_firewall_log(log_file_path)

```

Suspicious logs

```

Python 3.12.3 (tags/v3.12.3:f6650f9, Apr  9 2024, 14:05:25) [MSC v.1938 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

= RESTART: C:/workarea/test.py
Monitoring firewall log file: C:\Users\hiyas\OneDrive\Desktop\pfirewall.log1.txt
New entries detected!
ALERT: Detected suspicious traffic from 10.0.0.5 to 8.8.8.8
New Entry:
07 14:45:21 Block UDP 10.0.0.5 8.8.8.8 12345 53 512 FIN ACK 1 ICMP-
Error: An unexpected error occurred: 'Action'
|

```

Non-suspicious logs

```

===== RESTART: C:/workarea/test.py =====
Monitoring firewall log file: C:\Users\hiyas\OneDrive\Desktop\pfirewall.log1.txt
New entries detected!
New Entry:
2024-05-07 11:20:30 Allow UDP 10.1.1.2 54.32.19.8 5678 123 4096 FIN ACK 0 ICMP-
|

```

Integrated code

```

import os
import time

def parse_firewall_log_entry(log_entry):
    # Function to parse a single firewall log entry into a dictionary of fields
    fields = [
        'Date', 'Time', 'Action', 'Protocol', 'SourceIP', 'DestinationIP',
        'SourcePort', 'DestinationPort', 'Size', 'TCPFlags', 'TCPSYN', 'TCPACK',
        'TCPWin', 'ICMPType', 'ICMPCode'
    ]
    try:
        parsed_entry = dict(zip(fields, log_entry.split('\t')))
    except Exception as e:
        print(f"Error parsing log entry: {log_entry}")
        raise # Reraise the exception for detailed traceback
    return parsed_entry

def is_suspicious_entry(log_entry):
    # Function to determine if a firewall log entry is suspicious
    # Example criteria: Action is 'Block' and Protocol is 'UDP'
    return log_entry.get('Action') == 'Block' and log_entry.get('Protocol') == 'UDP'

def generate_alert(log_entry):
    # Function to generate alerts for suspicious firewall log entries
    source_ip = log_entry['SourceIP']
    destination_ip = log_entry['DestinationIP']
    print(f"ALERT: Detected suspicious traffic from {source_ip} to {destination_ip}")

def monitor_firewall_log(log_file_path):
    try:
        if not os.path.exists(log_file_path):
            print(f"Error: Log file '{log_file_path}' not found.")
            return

        print(f"Monitoring firewall log file: {log_file_path}")

        # Display existing logs
        print("Existing Logs:")
        with open(log_file_path, 'r') as log_file:
            for line in log_file:
                try:

```

```

                    # Display existing logs
                    print("Existing Logs:")
                    with open(log_file_path, 'r') as log_file:
                        for line in log_file:
                            try:
                                parsed_entry = parse_firewall_log_entry(line.strip())
                                print(parsed_entry)
                            except Exception as e:
                                print(f"Error processing log entry: {line.strip()} - {e}")

        # Get initial file size
        current_file_size = os.path.getsize(log_file_path)

        # Infinite loop for continuous monitoring
        while True:
            # Get current file size
            new_file_size = os.path.getsize(log_file_path)

            # Check if the file size has increased (new entries added)
            if new_file_size > current_file_size:
                print("New entries detected!")

                # Open the log file and read new entries
                with open(log_file_path, 'r') as log_file:
                    log_file.seek(current_file_size) # Move to the last read position
                    new_entries = log_file.read(new_file_size - current_file_size)

                # Split new entries into lines and parse each log entry
                for line in new_entries.splitlines():
                    try:
                        parsed_entry = parse_firewall_log_entry(line)
                        if is_suspicious_entry(parsed_entry):
                            generate_alert(parsed_entry) # Generate alert for suspicious entry
                        else:
                            print("New Entry:")
                            print(line)
                    except Exception as e:
                        print(f"Error processing log entry: {line.strip()} - {e}")

                # Update current file size
                current_file_size = new_file_size

```

```

        # Sleep for a short interval (e.g., 5 seconds) before checking again
        time.sleep(5)

    except Exception as e:
        print(f"Error: An unexpected error occurred: {e}")

if __name__ == "__main__":
    log_file_path = r'C:\Users\hiyas\OneDrive\Desktop\pfirewall.log1.txt'
    monitor_firewall_log(log_file_path)

```

Output:

```

{"Date": "2024-05-07", "Time": "10:35:20", "Action": "Allow", "Protocol": "ICMP", "SourceIP": "192.168.1.102", "DestinationIP": "8.8.8.8", "SourcePort": "0", "DestinationPort": "0", "Size": "64", "TCPFlags": "0", "TCPSYN": "0", "TCPACK": "0", "TCPWin": "0", "ICMPType": "Echo Request"},
{"Date": "2024-05-07", "Time": "12:15:30", "Action": "Allow", "Protocol": "TCP", "SourceIP": "192.168.1.103", "DestinationIP": "8.8.8.8", "SourcePort": "54321", "DestinationPort": "80", "Size": "2048", "TCPFlags": "ACK", "TCPSYN": "0", "TCPACK": "0", "TCPWin": "1", "ICMPType": "0"},
{"Date": "2024-05-07", "Time": "10:40:05", "Action": "Block", "Protocol": "TCP", "SourceIP": "192.168.1.103", "DestinationIP": "8.8.8.8", "SourcePort": "54321", "DestinationPort": "80", "Size": "2048", "TCPFlags": "ACK", "TCPSYN": "0", "TCPACK": "0", "TCPWin": "1", "ICMPType": "0"},
{"Date": "2024-05-07", "Time": "10:42:30", "Action": "Block", "Protocol": "UDP", "SourceIP": "192.168.1.104", "DestinationIP": "8.8.8.8", "SourcePort": "9876", "DestinationPort": "123", "Size": "256", "TCPFlags": "0", "TCPSYN": "0", "TCPACK": "0", "TCPWin": "0", "ICMPType": "0"},
{"Date": "2024-05-07", "Time": "10:45:55", "Action": "Block", "Protocol": "ICMP", "SourceIP": "192.168.1.105", "DestinationIP": "8.8.8.8", "SourcePort": "0", "DestinationPort": "0", "Size": "128", "TCPFlags": "0", "TCPSYN": "0", "TCPACK": "0", "TCPWin": "0", "ICMPType": "Destination Unreachable"},
{"Date": "2024-05-07", "Time": "10:30:45", "Action": "Allow", "Protocol": "TCP", "SourceIP": "192.168.1.100", "DestinationIP": "8.8.8.8", "SourcePort": "34567", "DestinationPort": "443", "Size": "1024", "TCPFlags": "SYN ACK", "TCPSYN": "1", "TCPACK": "1", "TCPWin": "65535", "ICMPType": "0"},
{"Date": "2024-05-07", "Time": "10:32:15", "Action": "Allow", "Protocol": "UDP", "SourceIP": "192.168.1.101", "DestinationIP": "8.8.8.8", "SourcePort": "12345", "DestinationPort": "53", "Size": "512", "TCPFlags": "0", "TCPSYN": "0", "TCPACK": "0", "TCPWin": "0", "ICMPType": "0"},
{"Date": "2024-05-07", "Time": "10:35:20", "Action": "Allow", "Protocol": "ICMP", "SourceIP": "192.168.1.102", "DestinationIP": "8.8.8.8", "SourcePort": "0", "DestinationPort": "0", "Size": "64", "TCPFlags": "0", "TCPSYN": "0", "TCPACK": "0", "TCPWin": "0", "ICMPType": "Echo Request"}

```

